# Network Utilization Monitor

SDN Mini-Project - POX + Mininet + OpenFlow 1.0

**4**
Hosts

**2**
Switches

**OpenFlow**
1.0

**POX**
Controller

Abhigyan Dutta | PES2UG24CS019

# Problem Statement & Objective

## The Problem

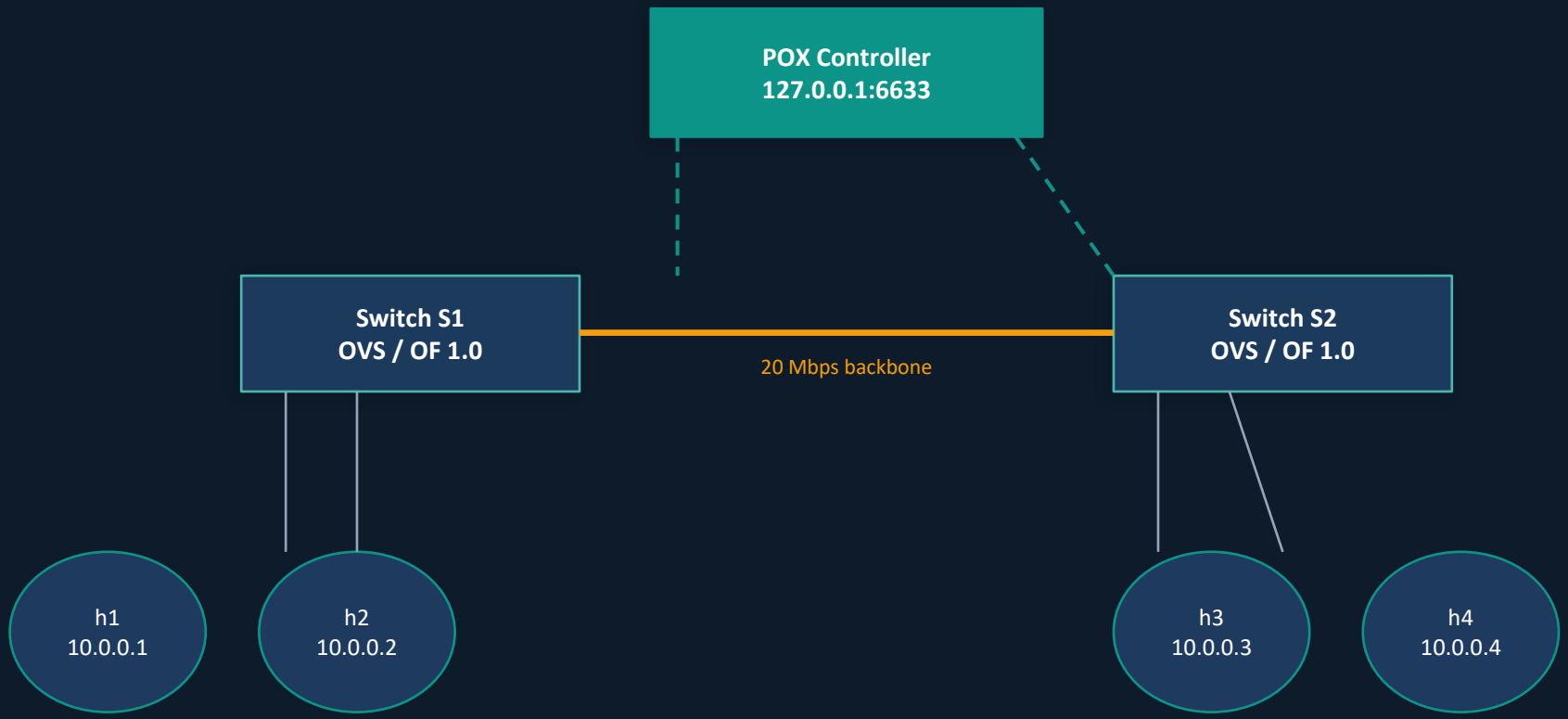
In traditional networks, per-link utilization is opaque - administrators have no real-time view of which flows consume bandwidth.

In SDN, the controller has global visibility. This project exploits that to build a live utilization monitor using OpenFlow statistics.

## Objectives

- Implement a MAC-learning switch with POX & OpenFlow flow rules
- Poll per-port statistics every 2 seconds from each switch
- Calculate real-time bandwidth utilization per port
- Alert when utilization exceeds threshold (1 Mbps)
- Demonstrate with iperf stress tests & Wireshark captures

# Topology & Architecture




10 Mbps host links · 20 Mbps inter-switch backbone · Dashed = OpenFlow control channel

# SDN Logic & Flow Rule Implementation

### ① Table-Miss Rule

Priority 0 catch-all installed at switch connection.
Unknown packets → OFPP_CONTROLLER.
Ensures all new flows reach POX for MAC learning.

### ② MAC Learning + Flow Rule

packet_in fires → learn src MAC→port mapping.
Dst MAC known: install priority-10 flow rule.
Dst unknown: flood once, no flow installed.

### ③ Stats Polling & Alert

Timer fires every 2s → send ofp_stats_request.
Receive port byte counters → compute Δbytes/Δt.
Alert logged if port exceeds 1 Mbps.

## Installed Flow Rule Structure

| Field | Value | Purpose |
|---|---|---|
| match | src_mac + dst_mac + in_port | Identify exact flow |
| priority | 10 (table-miss = 0) | Specific rules beat catch-all |
| idle_timeout | 30 s | Remove stale flows |
| hard_timeout | 120 s | Max rule lifetime |
| action | output → learned_port | Forward, not flood |

# Test Scenarios & Results

## 01

### Baseline Connectivity

**0% packet loss**

pingall across all 4 hosts.
Flow rules installed on first packet_in.
Subsequent pings bypass controller.

```
--- 10.0.0.3 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4171ms
rtt min/avg/max/mdev = 0.064/0.132/0.193/0.043 ms
```

## 02

### Single iperf Stream

**9.30 Mbits/sec**

h1 → h3 TCP for 15 s.
Near 10 Mbps link limit (91%).
POX stats detects utilization spike.

```
[  1] local 10.0.0.1 port 49700 connected with 10.0.0.3 port 5001 (icwnd/mss/irtt=14/1448/9113)
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-21.3008 sec  23.6 MBytes  9.30 Mbits/sec
```

## 03

### Flow Table & Port Dump

**Rules confirmed**

ovs-ofctl dump-flows s1 / s2.
ovs-ofctl dump-ports s1.
Packet & byte counters visible.

```
mininet> sh ovs-ofctl dump-flows s1
 cookie=0x0, duration=344.883s, table=0, n_packets=149, n_bytes=12321, priority=0 actions=CONTROLLER:65535
mininet> sh ovs-ofctl dump-flows s2
 cookie=0x0, duration=349.924s, table=0, n_packets=146, n_bytes=11903, priority=0 actions=CONTROLLER:65535
mininet> sh ovs-ofctl dump-ports s1
OFPST_PORT reply (xid=0x2): 4 ports
  port LOCAL: rx pkts=0, bytes=0, drop=0, errs=0, frame=0, over=0, crc=0
           tx pkts=0, bytes=0, drop=0, errs=0, coll=0
  port  "s1-eth1": rx pkts=1733, bytes=17811094, drop=0, errs=0, frame=0, over=0, crc=0
           tx pkts=1831, bytes=124377, drop=0, errs=0, coll=0
  port  "s1-eth2": rx pkts=43, bytes=3334, drop=0, errs=0, frame=0, over=0, crc=0
           tx pkts=122, bytes=11528, drop=0, errs=0, coll=0
  port  "s1-eth3": rx pkts=1799, bytes=120043, drop=0, errs=0, frame=0, over=0, crc=0
           tx pkts=1802, bytes=17811023, drop=0, errs=0, coll=0
mininet>
```

## 04

### Multi-host Parallel

**9.15+9.39 Mbps**

h2→h1 & h4→h3 simultaneously.
Both flow via 20 Mbps backbone.
POX logs both switch port stats.

```
[  1] local 10.0.0.4 port 47408 connected with 10.0.0.3 port 5001 (icwnd/mss/irtt=14/1448/5116)
[ ID] Interval       Transfer     Bandwidth
[  1] 0.0000-11.0517 sec  12.4 MBytes  9.39 Mbits/sec
mininet>
```

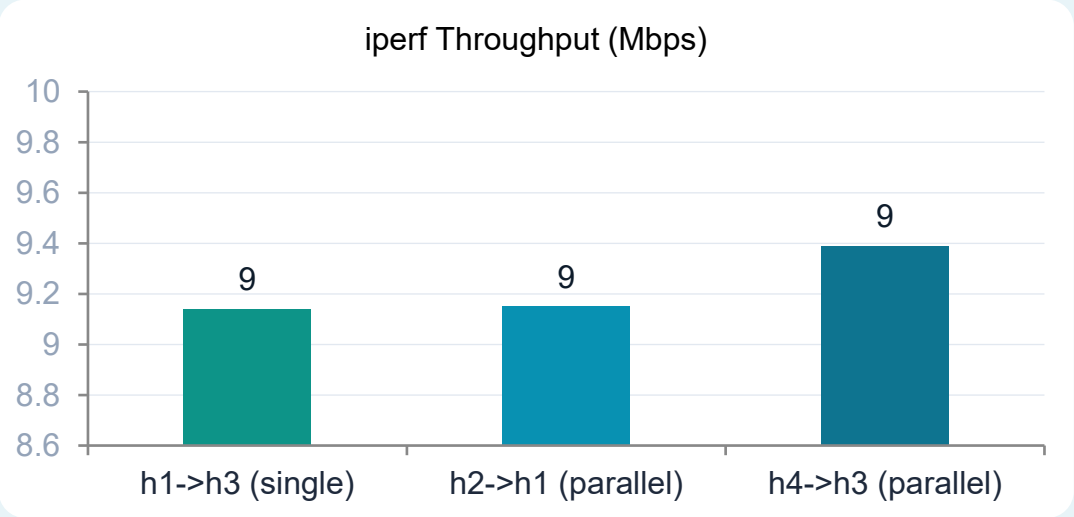
# Performance Observation & Analysis

**9.14**
Mbits/sec
h1→h3 single stream
(91% of 10 Mbps link)

**9.39**
Mbits/sec
h4→h3 parallel stream
(multi-host scenario)

**2s**
poll interval
Stats frequency
(OpenFlow stats request)

**0%**
packet loss
All scenarios
(pingall confirmed)



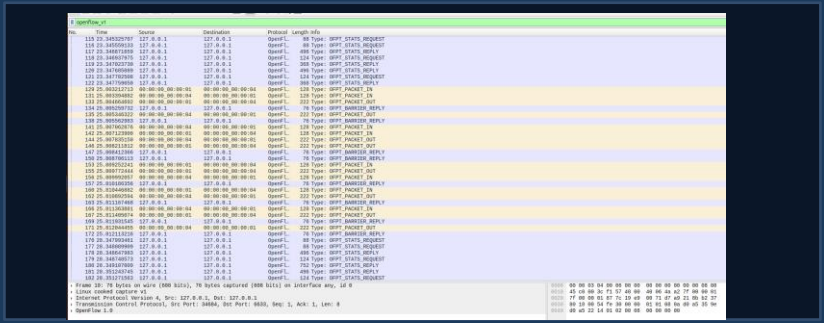

## Key Observations

- Streams approach the 10 Mbps TCLink limit — enforced correctly
- 20 Mbps backbone handles both parallel flows without saturation
- Flow rules eliminate repeated packet_in after first exchange
- Port stats Δbytes/Δt accurately reflects iperf bandwidth
- HIGH UTILIZATION ALERT fires above 1 Mbps threshold

# Proof of Execution — Wireshark Captures

## OpenFlow Control Plane

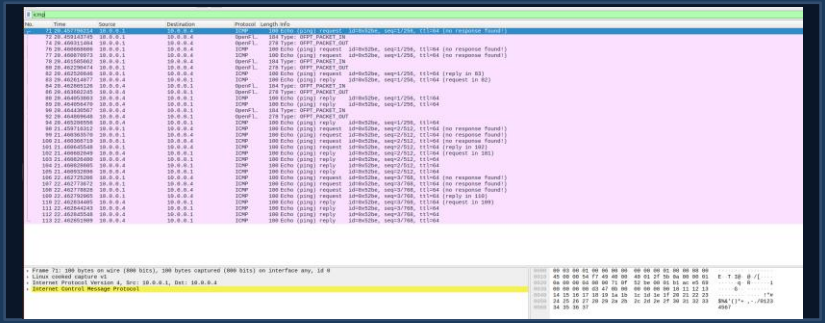
**Filter: openflow_v1**



- OFPT_STATS_REQUEST sent every 2s by POX
- OFPT_STATS_REPLY returns port byte counters
- OFPT_PACKET_IN seen on first ping (unknown dst)
- OFPT_PACKET_OUT follows immediately
- OFPT_BARRIER_REPLY confirms flow mod ack

## Data Plane — ICMP Traffic

**Filter: icmp**



- Echo Request / Reply pairs clearly visible
- First ping triggers PacketIn → controller
- Subsequent pings use installed flow rule
- OpenFlow PacketOut only on first exchange
- Confirms learning switch behaviour working

# POX Controller Output

```
----- FLOW TABLE: Switch 00-00-00-00-00-01 (1 entries) -----
INFO:network_monitor:  match=ofp_match
   | actions=[<output port:65533 max_len:65535>] | packets=139 | bytes=11432 | duration=179s
INFO:network_monitor:--------------------------------------------------
INFO:network_monitor:
========== PORT STATS: Switch 00-00-00-00-00-02 ==========
INFO:network_monitor:  Port  1 | RX:     0.00 Kbps | TX:     0.00 Kbps | RX pkts:     49 | TX pkts:    120 | Drops: 0/0
INFO:network_monitor:  Port  2 | RX:     0.00 Kbps | TX:     0.00 Kbps | RX pkts:     43 | TX pkts:    114 | Drops: 0/0
INFO:network_monitor:  Port  3 | RX:     0.00 Kbps | TX:     0.00 Kbps | RX pkts:     94 | TX pkts:     92 | Drops: 0/0
INFO:network_monitor:  Active flows installed: 75
INFO:network_monitor:==================================================
INFO:network_monitor:
----- FLOW TABLE: Switch 00-00-00-00-00-02 (1 entries) -----
INFO:network_monitor:  match=ofp_match
   | actions=[<output port:65533 max_len:65535>] | packets=136 | bytes=11214 | duration=179s
INFO:network_monitor:--------------------------------------------------
INFO:network_monitor:
========== PORT STATS: Switch 00-00-00-00-00-01 ==========
INFO:network_monitor:  Port  1 | RX:     0.00 Kbps | TX:     0.00 Kbps | RX pkts:     48 | TX pkts:    121 | Drops: 0/0
INFO:network_monitor:  Port  2 | RX:     0.00 Kbps | TX:     0.00 Kbps | RX pkts:     42 | TX pkts:    115 | Drops: 0/0
INFO:network_monitor:  Port  3 | RX:     0.00 Kbps | TX:     0.00 Kbps | RX pkts:     92 | TX pkts:     94 | Drops: 0/0
INFO:network_monitor:  Active flows installed: 75
INFO:network_monitor:==================================================
INFO:network_monitor:
----- FLOW TABLE: Switch 00-00-00-00-00-01 (1 entries) -----
INFO:network_monitor:  match=ofp_match
   | actions=[<output port:65533 max_len:65535>] | packets=139 | bytes=11432 | duration=184s
INFO:network_monitor:--------------------------------------------------
INFO:network_monitor:
========== PORT STATS: Switch 00-00-00-00-00-02 ==========
INFO:network_monitor:  Port  1 | RX:     0.00 Kbps | TX:     0.00 Kbps | RX pkts:     49 | TX pkts:    120 | Drops: 0/0
INFO:network_monitor:  Port  2 | RX:     0.00 Kbps | TX:     0.00 Kbps | RX pkts:     43 | TX pkts:    114 | Drops: 0/0
INFO:network_monitor:  Port  3 | RX:     0.00 Kbps | TX:     0.00 Kbps | RX pkts:     94 | TX pkts:     92 | Drops: 0/0
INFO:network_monitor:  Active flows installed: 75
INFO:network_monitor:==================================================
INFO:network_monitor:
----- FLOW TABLE: Switch 00-00-00-00-00-02 (1 entries) -----
INFO:network_monitor:  match=ofp_match
   | actions=[<output port:65533 max_len:65535>] | packets=136 | bytes=11214 | duration=184s
INFO:network_monitor:--------------------------------------------------
========== PORT STATS: Switch 00-00-00-00-00-01 ==========
INFO:network_monitor:  Port  1 | RX:  7480.20 Kbps | TX:    12.65 Kbps | RX pkts:    657 | TX pkts:    717 | Drops: 0/0
WARNING:network_monitor:  *** HIGH UTILIZATION ALERT: Switch 00-00-00-00-00-01 Port 1 RX=7.48 Mbps TX=0.01 Mbps ***
INFO:network_monitor:  Port  2 | RX:     0.00 Kbps | TX:     0.00 Kbps | RX pkts:     22 | TX pkts:     85 | Drops: 0/0
INFO:network_monitor:  Port  3 | RX:    12.65 Kbps | TX:  7480.20 Kbps | RX pkts:    688 | TX pkts:    689 | Drops: 0/0
WARNING:network_monitor:  *** HIGH UTILIZATION ALERT: Switch 00-00-00-00-00-01 Port 3 RX=0.01 Mbps TX=7.48 Mbps ***
INFO:network_monitor:  Active flows installed: 37
INFO:network_monitor:==================================================
```

## What the Output Shows

### Port Stats (every 2s)

- RX / TX Kbps per port
- Packet counts (RX + TX)
- Drop counts = 0 (healthy network)

### Flow Table Entries

- match=ofp_match shown
- actions=[output:port]
- packets= / bytes= / duration=

### Alert Line

- *** HIGH UTILIZATION ALERT ***
- Switch ID + Port + Mbps value
- Fires when port > 1 Mbps threshold

# Deliverables & GitHub Repository

github.com/UltraBot05/CN-Miniproject-SDN-Network-Monitor

### 1. Working Demo

- Live Mininet topology
- 4 hosts + 2 OVS switches
- POX controller active
- All 4 scenarios shown

### 2. Source Code

- network_monitor_topo.py
- network_monitor.py (ext)
- Clean, commented code
- Public GitHub repo

### 3. README.md

- Problem statement
- Setup + execution steps
- Expected output shown
- Tool versions listed

### 4. Screenshots

- POX port stats logs
- iperf throughput results
- ovs-ofctl flow dumps
- Wireshark captures

# Summary

## SDN Concept → Project Implementation

| SDN Concept | This Project |
|---|---|
| Control / Data Plane separation | POX (control) + OVS switches (data) |
| OpenFlow packet_in event | MAC learning + flow rule installation |
| Match-Action flow rules | Priority 10 rules: src+dst MAC -> output port |
| Flow statistics | ofp_stats_request -> per-port Kbps every 2s |
| Controller-defined forwarding | Table-miss -> controller; known flows -> direct |

Real-time SDN network utilization monitoring:

**Mininet · POX · Open vSwitch · iperf · Wireshark · OpenFlow 1.0**